

Project Phase 3 Report
Consultancy Appointment and Management System (CAMS)
Group 17 | Betül Merey 33963 Nehir Ceylan 35242

Database Description

CAMS is a relational database system designed to manage consultancy appointments between clients and professionals. It tracks services offered, locations used, payments made, and reviews submitted by clients. The schema consists of 7 tables: CLIENT, PROFESSIONAL, SERVICE, LOCATION, APPOINTMENT, PAYMENT (weak entity), and REVIEW (weak entity).

Query 1 — List All Clients (Nehir)

Description: Retrieve all rows and columns from the CLIENT table.

Category: A — Retrieve all rows

SQL:

```
SELECT * FROM CLIENT;
```

Relational Algebra:

```
CLIENT
```

Query 2 — Clients Born After 1990 (Nehir)

Description: List the name, email, and date of birth of clients born after January 1, 1990.

Category: A — Selection + Projection

SQL:

```
SELECT name, email, date_of_birth  
FROM CLIENT  
WHERE date_of_birth > '1990-01-01';
```

Relational Algebra:

```
 $\pi$  name, email, date_of_birth ( $\sigma$  date_of_birth > '1990-01-01' (CLIENT))
```

Query 3 — Appointments with Client Names (Betül)

Description: List each appointment ID and date along with the name of the client who booked it.

Category: B — Inner join between 2 tables

SQL:

```
SELECT A.appointment_id, A.date, C.name AS client_name
FROM APPOINTMENT A
JOIN CLIENT C ON A.client_id = C.client_id;
```

Relational Algebra:

```
 $\pi$  appointment_id, date, CLIENT.name
( APPOINTMENT  $\bowtie$  APPOINTMENT.client_id = CLIENT.client_id CLIENT )
```

Query 4 — Detailed Appointment List (Betül)

Description: List each appointment showing the date, client name, professional name, and service name.

Category: B — Join between 3 tables

SQL:

```
SELECT A.appointment_id, A.date,
       C.name AS client_name,
       P.name AS professional_name,
       S.service_name
FROM APPOINTMENT A
JOIN CLIENT C ON A.client_id = C.client_id
JOIN PROFESSIONAL P ON A.professional_id = P.professional_id
JOIN SERVICE S ON A.service_id = S.service_id;
```

Relational Algebra:

```
 $\pi$  appointment_id, date, CLIENT.name, PROFESSIONAL.name, service_name
(
  ( (APPOINTMENT  $\bowtie$  APPOINTMENT.client_id = CLIENT.client_id CLIENT)
     $\bowtie$  APPOINTMENT.professional_id = PROFESSIONAL.professional_id PROFESSIONAL)
     $\bowtie$  APPOINTMENT.service_id = SERVICE.service_id SERVICE
)
```

Query 5 — All Services Ordered by Name (Betül)

Description: List all services sorted in ascending alphabetical order by service name.

Category: D — ORDER BY ascending

Note: Sorting is outside basic RA scope; the RA expression shows the queried relation.

SQL:

```
SELECT service_id, service_name, description
FROM SERVICE
ORDER BY service_name ASC;
```

Relational Algebra:

```
 $\pi$  service_id, service_name, description (SERVICE)
```

Query 6 — Completed Payments Ordered by Amount (Nehir)

Description: List all completed payments sorted from highest to lowest amount.

Category: D — ORDER BY descending

Note: Sorting is outside basic RA scope; the RA expression shows the filtered relation.

SQL:

```
SELECT payment_id, appointment_id, amount, payment_method
FROM PAYMENT
WHERE payment_status = 'Completed'
ORDER BY amount DESC;
```

Relational Algebra:

```
 $\pi$  payment_id, appointment_id, amount, payment_method
  ( $\sigma$  payment_status = 'Completed' (PAYMENT))
```

Query 7 — Count of Appointments per Service (Betül)

Description: Count how many appointments have been made for each service.

Category: C — COUNT with GROUP BY

Note: Aggregate functions are outside basic RA scope; the RA expression shows the joined relation.

SQL:

```
SELECT S.service_name, COUNT(A.appointment_id) AS total_appointments
FROM SERVICE S
JOIN APPOINTMENT A ON S.service_id = A.service_id
GROUP BY S.service_name;
```

Relational Algebra:

```
 $\pi$  service_name, appointment_id ( SERVICE  $\bowtie$  SERVICE.service_id =
APPOINTMENT.service_id APPOINTMENT )
```

Query 8 — Average Payment Amount per Method (Nehir)

Description: Calculate the average payment amount grouped by payment method.

Category: C — AVG with GROUP BY

SQL:

```
SELECT payment_method, AVG(amount) AS avg_amount
FROM PAYMENT
GROUP BY payment_method;
```

Relational Algebra:

```
 $\pi$  payment_method, amount (PAYMENT)
```

Query 9 — Clients Who Have Not Left a Review (Betül)

Description: Find client IDs of clients who have had an appointment but submitted no review.

Category: E — Set Difference

SQL:

```
SELECT DISTINCT client_id FROM APPOINTMENT WHERE appointment_id NOT IN (SELECT
appointment_id FROM REVIEW);
```

Relational Algebra:

```
 $\pi$  client_id (APPOINTMENT) -  $\pi$  client_id (APPOINTMENT  $\bowtie$  REVIEW)
```

Query 10 — Clients with at Least One Completed Payment (Nehir)

Description: List clients who have made at least one payment with status Completed.

Category: E — EXISTS

SQL:

```
SELECT C.name, C.email
FROM CLIENT C
WHERE EXISTS (
    SELECT 1
    FROM APPOINTMENT A
    JOIN PAYMENT P ON A.appointment_id = P.appointment_id
    WHERE A.client_id = C.client_id
    AND P.payment_status = 'Completed'
);
```

Relational Algebra:

```
 $\pi$  name, email
(
    CLIENT  $\bowtie$  CLIENT.client_id =
        ( $\pi$  client_id
        ( $\sigma$  payment_status = 'Completed'
```

```

        (APPOINTMENT ⋈ APPOINTMENT.appointment_id = PAYMENT.appointment_id
PAYMENT)
    )
    ).client_id
)

```

Appendix- AI Usage Documentation

Tools: Gemini, Claude, ChatGPT

Uploaded files: phase01.pdf, phase02.pdf, Group17_Phase2.sql, phase_03.pdf

Initial Prompt (Gemini & Claude): Here phase1 and 2 files are our project you can see what we have done. According to them we want you to do phase03 (be strict about the requirements)

Verification Prompt (ChatGPT): "On pasted file you can see our project. Can you check their correctness and if they are ok for the requirements or not? If there are any false can you correct it."

AI Responses:

- Gemini and Claude provided the initial population scripts and query structures.
- ChatGPT provided feedback, stating: *"I went through both your project requirements (PDF) and your queries carefully. Overall, you're actually in a good position—your SQL is mostly correct and your coverage of categories is solid. But there are some important mistakes in Relational Algebra (RA) and a few conceptual issues that you should fix to avoid losing points."*

Implementation Strategy (Used/Modified/Rejected):

- **Directly Used:** The **INSERT** statements were used directly to populate the database with consistent test data.
- **Modified:** The initial Relational Algebra (RA) results from Gemini/Claude were significantly modified because they contained notations not used in our course. We reformatted them to strictly use standard notations ($\sigma, \pi, \bowtie, \dots$).